

Big 5 Image Recognition Explanation



A project by Mihir Arjun

Table of Contents

Table of Contents	2
Image Recognition for Classifying the Big Five Animals of South Africa	3
Data Preparation and Cleaning	3
Loading and Visualising the Data	3
Data Scaling	3
Normalising Pixel Values	3
Verifying Scaling	3
Splitting the Data	4
Building the Deep Learning Model	4
Initialising the Model	4
Adding Convolutional and Pooling Layers	4
Flattening and Adding Dense Layers	4
Adding the Output Layer	4
Compiling the Model	4
Training the Model	5
Fitting the Model	5
Saving the Trained Model	5
Plotting Model Performance	5
Plotting Loss	5
Plotting Accuracy	6
Evaluating the Model	6
Assessing Performance on Test Data	6
Evaluation Process	6
Testing the Model	6
Loading the Saved Model	6
Preparing a Test Image	6
Making Predictions	6
Determining the Predicted Class	7
Extracting Prediction Probability	7
Applying a Confidence Threshold	7
Outputting the Result	7
Summary	7

Image Recognition for Classifying the Big Five Animals of South Africa

Image classification is a pivotal task in the field of computer vision, aiming to assign labels to images based on their content. This project focuses on classifying images of the Big Five animals of South Africa using a Convolutional Neural Network (CNN) built with TensorFlow and Keras. The Big Five includes buffalo, elephant, leopard, lion, and rhino. This guide provides a step-by-step approach to building, training, and deploying a CNN for this classification task.

Data Preparation and Cleaning

Iterate through each subdirectory corresponding to an animal class and perform the following actions:

1. **Image Validation:** For each image, attempt to read it using OpenCV to ensure it's not corrupted.
2. **Format Verification:** Check the image file type against the list of supported formats.
3. **Removal of Invalid Images:** If an image doesn't match the supported formats or is corrupted, remove it from the dataset.

This cleaning process ensures that the dataset is free from invalid or unsupported images, which could otherwise hinder model performance.

Loading and Visualising the Data

Utilise TensorFlow's `image_dataset_from_directory` utility to load images from the specified directory. This function automatically infers labels based on subdirectory names and organises the data into batches, facilitating efficient training.

Optionally, visualise a few sample images from each class to verify that the images are correctly loaded and labelled. This step helps in ensuring the integrity of the dataset before proceeding to model training.

Data Scaling

Normalising Pixel Values

Normalise the pixel values of the images to a range between 0 and 1. This scaling is achieved by dividing the pixel values by 255.0, which helps in accelerating the training process and improving model performance by ensuring that all input features contribute equally.

Verifying Scaling

After normalisation, fetch a new batch of images and visualise them to confirm that the scaling has been applied correctly. This step is crucial for verifying that the data preprocessing pipeline is functioning as intended.

Splitting the Data

Split the dataset into three distinct sets:

1. **Training Set (70%):** Used to train the model.
2. **Validation Set (20%):** Used to tune hyperparameters and monitor the model's performance during training, helping to prevent overfitting.
3. **Test Set (10%):** Used to evaluate the final model's performance on unseen data.

Building the Deep Learning Model

Initialising the Model

Create a Sequential model using Keras, which allows stacking layers linearly. This simplicity is suitable for many standard neural network architectures.

Adding Convolutional and Pooling Layers

1. **Convolutional Layers:** Extract spatial features from the images by applying filters. The number of filters and kernel sizes are defined to capture various levels of detail.
2. **Activation Function:** ReLU (Rectified Linear Unit) introduces non-linearity, enabling the model to learn complex patterns.
3. **MaxPooling Layers:** Downsample the feature maps, reducing computational complexity and controlling overfitting by summarising feature information.

Flattening and Adding Dense Layers

After feature extraction, flatten the 2D feature maps into a 1D vector and add fully connected (Dense) layers:

1. **Flatten Layer:** Converts the multi-dimensional feature maps into a single vector.
2. **Dense Layer:** Adds a layer with a specified number of neurons (e.g., 256) to learn high-level representations.

Adding the Output Layer

For multi-class classification, add a Dense layer with a number of neurons equal to the number of classes (5 in this case) and apply the softmax activation function. Softmax ensures that the output probabilities sum to 1, making it suitable for classification tasks.

Compiling the Model

Define the optimizer, loss function, and evaluation metrics:

- **Optimizer:** Adam optimizer is chosen for its efficiency and adaptability.
- **Loss Function:** Sparse Categorical Cross Entropy is suitable for integer-labelled multi-class classification.
- **Metrics:** Accuracy is monitored to assess the proportion of correctly classified instances during training.

Training the Model

Fitting the Model

Train the CNN using the training dataset over a specified number of epochs (e.g., 20). The validation set is used concurrently to monitor the model's performance and prevent overfitting. The TensorBoard callback logs training metrics for visualisation.

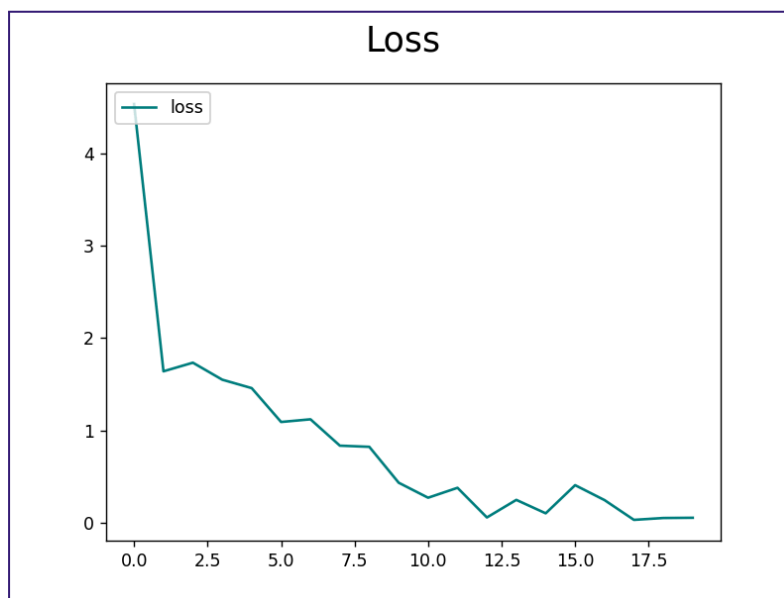
Saving the Trained Model

After training, save the model to the designated directory. This allows for future use without retraining and facilitates model deployment.

Plotting Model Performance

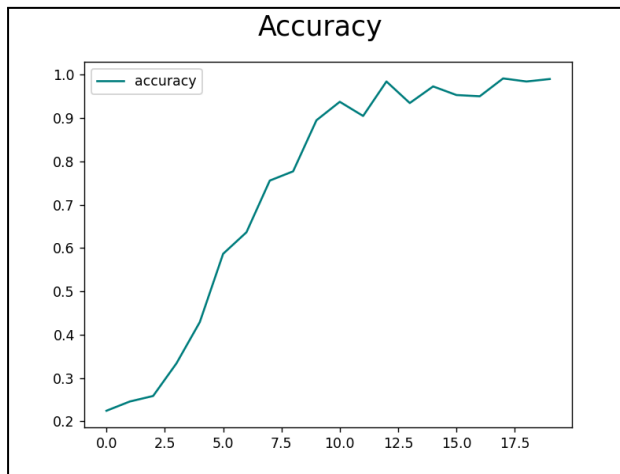
Plotting Loss

Visualise the training and validation loss over epochs to assess how well the model is minimising errors. A decreasing trend indicates effective learning, while divergence between training and validation loss may signal overfitting.



Plotting Accuracy

Plot the training and validation accuracy to evaluate the model's ability to correctly classify images. Consistent improvement in accuracy suggests successful learning, whereas stagnation or decline may require model adjustments.



Evaluating the Model

Assessing Performance on Test Data

Evaluate the trained model's performance using the test dataset, which comprises unseen data. Calculate metrics such as Sparse Categorical Accuracy to determine the model's effectiveness in real-world scenarios.

Evaluation Process

Iterate through the test data in batches, make predictions, and update the accuracy metric accordingly. This provides an overall accuracy score reflecting the model's capability to generalise to new data.

Testing the Model

Loading the Saved Model

Retrieve the saved model from the storage directory to make predictions on new images without retraining.

Preparing a Test Image

To test the model, load a new image, resize it to match the model's input dimensions (e.g., 256x256 pixels), and normalise its pixel values. This ensures compatibility with the trained model.

Making Predictions

Use the loaded model to predict the class probabilities of the test image. The model outputs a probability distribution across the defined classes.

Determining the Predicted Class

Identify the class with the highest probability as the predicted label. This involves finding the index of the maximum probability and mapping it to the corresponding class name.

Extracting Prediction Probability

Retrieve the probability associated with the predicted class to gauge the model's confidence in its prediction.

Applying a Confidence Threshold

Define a threshold (e.g., 60%) to determine whether the prediction is conclusive. If the highest probability meets or exceeds the threshold, accept the prediction; otherwise, indicate that the prediction is not conclusive.

Outputting the Result

Display the predicted class along with its probability if it meets the confidence threshold. If not, notify the user that the prediction lacks sufficient confidence, providing the highest probability achieved.

Summary

- **Data Cleaning:** Ensuring the dataset is free from invalid or unsupported images.
- **Data Loading and Scaling:** Efficiently loading images and normalising pixel values for optimal training.
- **Data Splitting:** Dividing the dataset into training, validation, and test sets to evaluate model performance accurately.
- **Model Building:** Constructing a CNN with convolutional, pooling, and dense layers tailored for multi-class classification.
- **Training and Saving:** Training the model while monitoring performance and saving the trained model for future use.
- **Performance Visualisation:** Plotting loss and accuracy to assess learning behaviour.
- **Model Evaluation and Testing:** Evaluating the model on unseen data and making predictions on new images with confidence thresholding.